

第 1 次作業-作業-HW1

學號：112111207

姓名：陳品霖

作業撰寫時間：100 (mins · 包含程式撰寫時間)

最後撰寫文件日期：2024/10/09

本份文件包含以下主題：

- ☒ 說明內容
- ☒ 個人認為完成作業須具備觀念

1. 請解釋何謂 `git` 中下列指令代表什麼？並舉個例子，同時必須說明該例子的結果。其指令有 `add`、`commit`、`push`、`fetch`、`pull`、`branch`、`checkout` 與 `merge`。

Ans:

1. `git add` = 將改動的文件加入到暫存區

範例：

```
git add file.txt
```

結果：以上指令會將 `file.txt` 加入到暫存區，準備提交。

2. `git commit` = 將暫存區的變更提交到本地儲存庫。

範例：

```
git commit -m "Added new feature"
```

結果：以上指令會將所有暫存區的改動保存到本地，並附上提交訊息「Added new feature」。

3. `git push` = 將本地提交推送到遠端儲存庫

範例：

```
git push origin main
```

結果：以上指令會把本地的 `main` 分支提交推送到遠端的 `origin` 分支。

4. `git fetch` = 從遠端儲存庫拉取更新，但不合併到當前分支

範例：

```
git fetch origin
```

結果：以上指令會從 **origin** 獲取最新變更，但不會自動合併。

5. **git pull** = 從遠端儲存庫獲取更新並自動合併到當前分支

範例：

```
git pull origin main
```

結果：以上指令會從遠端 **origin** 的 **main** 分支拉取最新改動並合併到當前分支。

6. **git branch** = 創建或列出本地分支

範例：

```
git branch new-feature
```

結果：以上指令將會創建一個名為 **new-feature** 的分支。

列出分支：

```
git branch
```

結果：列出所有本地分支，標記當前所在的分支。

7. **git checkout** = 切換到指定的分支或檢出某個提交

範例：

```
git checkout new-feature
```

結果：切換到 **new-feature** 分支，並進入該分支進行修改。

8. **git merge** = 合併另一個分支的改動到當前分支

範例：

```
git merge new-feature
```

結果：將 `new-feature` 的改動合併到當前分支。如果有衝突，需要手動解決。

2. 於專案下的檔案`**hw1.py**`，撰寫註解，以說明該程式每列中之背後意義。

Ans:

```
from typing import List
#從標準庫的 typing 模組中導入 List 類型。

def countLetters(sentence: str) -> List[int]:
    # 定義一個名為 countLetters 的函數，參數是 sentence (類型為字串)，返回值是整數列表。

    letterCount: List[int] = [0] * 26
    # 定義 letterCount 變數，其類型為整數列表，並初始化為 26 個 0，
    # 代表英文字母 a 到 z 的出現次數，初始值為 0。

    for char in sentence:
        # 使用 for 迴圈逐一檢查 sentence 中的每個字符。

        if char.isalpha():
            # 用 Python 內建方法 isalpha 檢查 char 是否為字母。
            index = ord(char) - ord('a')
            # 用 ord 函數將字符轉換為 ASCII 值，然後減去 'a' 的 ASCII 值
            # 得出該字母在字母表中的索引 (a=0, b=1, ..., z=25)。
            letterCount[index] += 1
            # 根據計算出的索引，將 letterCount 中對應字母的計數加 1。

    return letterCount
#回傳letterCount的值
pass
# `pass` 在這裡其實沒有實際作用，可以省略

def printLetterCount(letterCount: List[int]) -> None:
    # 定義一個名為 printLetterCount 的函式，參數是 letterCount
    # 參數 letterCount 是一個長度為 26 的整數列表，表示各字母的出現次數
    # 該函式不返回任何值，只是將每個字母及其出現次數打印出來

    for i in range(26):
        # 使用 for 迴圈檢查 0 到 25 的索引，檢查字母 a ~ z
        if letterCount[i] > 0:
            # 如果 letterCount 中的第 i 個元素大於 0，說明該字母至少出現過一次
            print(f"{chr(i + ord('a'))}: {letterCount[i]}")
            # 將索引 i 對應的字母 (使用 chr 函數將 i 加上 'a' 的 ASCII 值轉為字母) (就能打印出對應的字母)
            # 以及該字母的出現次數 (letterCount[i]) 打印出來

    pass
# `pass` 在這裡其實沒有實際作用，可以省略
```

```
inputSentence: str = "this is an apple"
# 定義 inputSentence 變數，類型為字串，內容為 "this is an apple" (可以任意改成你想要檢查的字串)
letterCount: List[int] = countLetters(inputSentence)
# 調用 countLetters 函式，將上面的inputSentence 作為參數，得到該句子中每個字母的出現次數
# letterCount 是整數列表，表示各個字母的出現次數
printLetterCount(letterCount)
# 調用 printLetterCount 函式，將 letterCount 列表傳入
# 以打印出句子中各字母的出現次數
```
```

---

```
inputSentence: str = "this is an apple"
定義 inputSentence 變數，類型為字串，內容為 "this is an apple" (可以任意改成你想要檢查的字串)
letterCount: List[int] = countLetters(inputSentence)
調用 countLetters 函式，將上面的inputSentence 作為參數，得到該句子中每個字母的出現次數
letterCount 是整數列表，表示各個字母的出現次數
printLetterCount(letterCount)
調用 printLetterCount 函式，將 letterCount 列表傳入
以打印出句子中各字母的出現次數
```

3. 請新增檔案\*\*hw1\_2.py\*\*，\*\*輸入一個正整數(N)，其中 $1 \leq N \leq 100000$ ，請將該正整數輸出進行反轉

Ans :

```
def reverse_number(N: int) -> int:
 # 檢查 N 是否在範圍內
 if N < 1 or N > 100000:
 return "輸入的數字不在範圍內"

 reversed_number = 0
 while N > 0:
 digit = N % 10 # 取出最後一位數字
 reversed_number = reversed_number * 10 + digit # 將反轉的數字加上這一位
 N = N // 10 # 移除最後一位數字
 return reversed_number

主程式部分
N = int(input("請輸入一個正整數 (1 ≤ N ≤ 100000): "))
print(reverse_number(N))
```

程式碼解釋:

這段程式碼檢查輸入的數字  $N$  是否在 1 和 100000 之間。如果  $N$  不符合範圍條件，則函式返回一個錯誤訊息 "輸入的數字不在範圍內"，並且不再繼續後面的反轉邏輯。

這個 while 迴圈會一直執行，直到  $N$  被處理完為止。它的邏輯是：

1.  $\text{digit} = N \% 10$ ：使用取模運算  $\%$  來提取數字的最後一位。如果  $N=123$  那麼  $\text{digit}$  會是 3。
2.  $\text{reversed\_number} = \text{reversed\_number} * 10 + \text{digit}$ ：將  $\text{reversed\_number}$  乘以 10，並將剛提取出的數字  $\text{digit}$  加到  $\text{reversed\_number}$  中。這一步逐步構建反轉的數字。例如：

- 第一次： $\text{reversed\_number} = 0 * 10 + 3 = 3$
- 第二次： $\text{reversed\_number} = 3 * 10 + 2 = 32$
- 第三次： $\text{reversed\_number} = 32 * 10 + 1 = 321$

3.  $N = N // 10$ ：使用整數除法  $//$  移除數字的最後一位。例如，如果  $N=123$ ，這一步後  $N$  變成 12。

---

4. **[課外題]**：請找尋資料，說明何謂**單元測試**，請新增檔案 **hw1\_3.py**，並利用溫度計攝氏轉華氏撰寫單元測試。

Ans:

什麼是單元測試？

單元測試 (Unit Testing) 是一種軟體測試方法，用來檢查單個功能單元 (例如一個函式或一個模組) 的正確性。單元測試的目標是確保每個單元獨立運作，並且能在不同情境下返回正確結果。它通常會針對一個模組中的多個邊界條件和正常條件進行測試。

```
def celsius_to_fahrenheit(celsius: float) -> float:
 """將攝氏溫度轉換為華氏溫度"""
 if not isinstance(celsius, (int, float)):
 raise ValueError("輸入的溫度必須是數字")
 return celsius * 9/5 + 32

單元測試部分
import unittest
import random
class TestTemperatureConversion(unittest.TestCase):

 def test_positive_temp(self):
 """測試正數溫度轉換"""
 a = random.randint(1, 1000)
 b = random.randint(1, 10000)
 c = random.uniform(1, 1000)

 result = celsius_to_fahrenheit(a)
 print(f"1. 攝氏 {a} 度轉換為華氏: {result} 度")

 result = celsius_to_fahrenheit(b)
 print(f"2. 攝氏 {b} 度轉換為華氏: {result} 度")

 result = celsius_to_fahrenheit(c)
 print(f"3. 攝氏 {c} 度轉換為華氏: {result} 度")
```

```
def test_negative_temp(self):
 """測試負數溫度轉換"""
 a = random.randint(-1000, -1)
 b = random.uniform(-1000, -1)
 result = celsius_to_fahrenheit(a)
 print(f"4. 攝氏 {a} 度轉換為華氏: {result} 度")
 result = celsius_to_fahrenheit(b)
 print(f"5. 攝氏 {b} 度轉換為華氏: {result} 度")

def test_zero(self):
 """測試零度轉換"""
 result = celsius_to_fahrenheit(0)
 print(f"6. 攝氏 0 度轉換為華氏: {result} 度")

def test_fractional_temp(self):
 """測試小數溫度"""
 a = random.random()
 result = celsius_to_fahrenheit(a)
 print(f"7. 攝氏 {a} 度轉換為華氏: {result} 度")

def test_edge_cases(self):
 """測試極端邊界情況"""
 result = celsius_to_fahrenheit(1e6)
 print(f"8. 攝氏 1e6 度轉換為華氏: {result} 度")
 result = celsius_to_fahrenheit(-1e6)
 print(f"9. 攝氏 -1e6 度轉換為華氏: {result} 度")

執行測試
if __name__ == '__main__':
 unittest.main()
```

### 程式碼解釋:

這個程式碼，能夠有效地將攝氏溫度轉換為華氏溫度，同時也能夠透過隨機測試來檢驗轉換功能的正確性。透過使用 `unittest` 和 `random` 模組，程式提供了全面的測試範圍，下方介紹程式的測試內容。

### 測試類別 `TestTemperatureConversion` 的功能

#### `test_positive_temp` :

- 測試隨機生成的正數攝氏溫度（範圍 1 到 1000 和 1 到 10000 的整數，以及 1 到 1000 的浮點數）。將每個隨機生成的攝氏溫度轉換為華氏，並打印結果。

#### `test_negative_temp` :

- 測試隨機生成的負數攝氏溫度（範圍 -1000 到 -1 的整數和浮點數）。將每個隨機生成的攝氏溫度轉換為華氏，並打印結果。

### test\_zero :

- 測試攝氏 0 度的轉換。打印攝氏 0 度轉換為華氏的結果。

### test\_fractional\_temp :

- 測試隨機生成的小數攝氏溫度（範圍 0 到 1 的隨機數）。將攝氏溫度轉換為華氏並打印結果。

### test\_edge\_cases :

- 測試攝氏極端溫度（例如 1,000,000 和 -1,000,000 的轉換）。打印攝氏極端溫度轉換的結果。

---

## 個人認為完成作業須具備觀念

需知道(迴圈 loop),(類別class),(繼承 Inheritance),(函式 Function),數據結構 (list,Tuple,Dictionary,set) · 以及物件導向程式設計 (Object-Oriented Programming, OOP),全域變數,區域變數的概念 · 還有一些python內建的方法以及函式庫。